



deti

universidade de aveiro
departamento de eletrónica,
telecomunicações e informática

Exploring Music Trends WS

Universidade de Aveiro

Ângela Ribeiro (109061), António Alberto (114622),

Hugo Castro (113889)

Index

Introduction.....	3
Data, Sources and Transformation.....	3
Data Sources.....	3
Billboard Hot 100 (digital.csv).....	3
Spotify Tracks Detailed Dataset (spotify-tracks-dataset-detailed.csv).....	3
RDF Model.....	3
Normalization and Transformation.....	3
Textual normalization.....	3
Artist splitting.....	4
Cross-dataset matching.....	4
RDF File Generation.....	4
Data Operations (SPARQL).....	4
Base Prefixes.....	4
Read Operations (SELECT).....	5
Dashboard — general counts.....	5
Dashboard — top artists by chart presence.....	5
Songs Explorer — filtered and paginated retrieval.....	5
Insights — party score.....	5
Advanced analytical queries used in Insights.....	6
Update Operations (INSERT / DELETE / DELETE+INSERT).....	7
INSERT — add alternative genre to a song.....	7
DELETE WHERE — remove featured artist relationship.....	7
DELETE + INSERT — replace an attribute value safely.....	7
Validation and Quality Queries.....	7
Implementation Notes.....	7
Application Features (UI).....	8
Dashboard (/).....	8
Songs Explorer (/songs/).....	8
Song Detail (/songs/<id>/?uri=...).....	8
Artist Detail (/artists/<id>/?uri=...).....	8
SPARQL Operations (/operations/).....	8
Insights (/insights/).....	9
Billboard (/billboard/).....	9
About Data (/about-data/).....	9
Conclusions.....	9
Setup to Run the Application.....	10

Introduction

This work describes the development of a web-app about popular music. It is focused on songs that appeared in digital music charts and their characteristics. The theme was chosen for the richness of relationships that naturally exist between its entities: songs, artists, collaborations/features, albums, genres, and chart positions, which we thought was well-suited for representation in triples. Our app aims to show and manage information about songs and artists, allowing users to explore popularity trends, audio features of tracks, and relationships between artists.

Data, Sources and Transformation

Data Sources

The system integrates two datasets from Kaggle.

Billboard Hot 100 (digital.csv)

This is the main dataset, and contains weekly records of the Hot 100 Billboard since 2004\.. The rows in this dataset all include the fields *Song*, *Artist*, *Rank*, *Weeks in Charts*, and *Date*. We use this to create our base songs, artists, and chart entries, that we then enrich with information from the other dataset.

Spotify Tracks Detailed Dataset (spotify-tracks-dataset-detailed.csv)

This dataset serves as semantic enrichment for the songs present in the first dataset. It contains detailed audio features for tracks, including *danceability*, *energy*, *valence*, *tempo*, *loudness*, *speechiness*, *acousticness*, *instrumentalness*, *liveness*, *popularity*, *duration_ms*, *explicit*, *album_name*, and *track_genre*.

RDF Model

The data is organized around three main entity types, each with its own namespace:

``http://music.org/type/Song`` represents a unique song;

``http://music.org/type/Artist`` represents an artist;

``http://music.org/type/ChartEntry`` represents a song's appearance in a chart on a specific date, allowing the same song to have multiple entries over time (e.g., "All I Want for Christmas Is You" by Mariah Carey with over 170 chart entries).

Predicates are defined under the namespace ``http://music.org/pred/`` and include structural relations (*mainArtist*, *featuredArtist*) as well as literal attributes (*name*, *rank*, *weeks*, *date*, *genre*, *albumName*, and the Spotify audio features).

URIs are generated deterministically via an MD5 hash of the normalized song name concatenated with the normalized main artist name, so that songs with the same title by different artists have different URIs.

Normalization and Transformation

Transforming the data into RDF required three main processing stages.

Textual normalization.

We normalized song and artist names before any matching or comparison operation. They are converted to lowercase, parenthetical expressions are removed, suffixes following dashes are stripped, and non-alphanumeric characters are eliminated. This normalization is used exclusively for matching between the two datasets and generating the URI.

During development, we found that over-aggressive normalization could merge distinct variants too early. To avoid this, the normalized form is used only as an internal matching key, while the original labels are preserved for display and RDF literals.

Artist splitting

The *Artist* field in the Billboard Hot 100 dataset shows a lot of variability in how collaborations are represented, like *featuring*, *feat.*, *ft.*, *with*, *and*, *&*, *+*, *;* and */*. The script implements a splitting pipeline that first replaces collaboration markers with commas, then splits the resulting string into tokens, and then handles edge cases. The first resulting token is treated as the main artist and the remaining ones are treated as featured artists.

This stage required specific fixes for false splits in artist names that are not collaborations. In particular, we had to protect band/duo names such as "Selena Gomez & The Scene" and "Dan + Shay" from being split into separate artists. Without this protection, the graph would incorrectly create extra featured artists and distort both collaboration metrics and artist pages.

We also aligned this logic with matching by maintaining a protected-name strategy for known patterns that contain connectors as part of the official name.

Cross-dataset matching

To enrich Billboard songs with Spotify audio attributes, we perform a fuzzy matching step between the two datasets. For each Billboard song, candidate tracks in the Spotify dataset are selected. The matching score combines text similarity of the song title with overlap between normalized artist tokens.

One of the main issues we fixed here was cross-artist contamination: tracks with similar names could be matched to the wrong artist when title similarity was high. We solved this by strengthening artist-overlap requirements in candidate selection, which prevented mismatches such as metadata from one artist leaking into another artist's song.

After these fixes, enrichment became more consistent and safer for songs with common titles or many remixes/versions.

RDF File Generation

The graph is built using the *rdflib* library, serialized in Turtle format, and stored in the file *music.ttl*. We then imported it into the GraphDB triplestore.

Data Operations (SPARQL)

This section describes the SPARQL operations used in the application. They are divided into read operations for UI pages and analytics, update operations for data management, and validation operations for data quality.

Base Prefixes

```
PREFIX pred: <http://music.org/pred/>
PREFIX type: <http://music.org/type/>
```

```
PREFIX xsd: <http://www.w3.org/2001/XMLSchema#>
```

Read Operations (SELECT)

Dashboard — general counts

```
SELECT (COUNT(?a) AS ?count)
WHERE { ?a a type:Artist . }

SELECT (COUNT(?s) AS ?count)
WHERE { ?s a type:Song . }

SELECT (COUNT(?e) AS ?count)
WHERE { ?e a type:ChartEntry . }
```

Dashboard — top artists by chart presence

```
SELECT ?artist ?artistName (COUNT(?entry) AS ?entries)
WHERE {
  ?entry a type:ChartEntry ; pred:song ?song .
  ?song pred:mainArtist ?artist .
  ?artist pred:name ?artistName .
}
GROUP BY ?artist ?artistName
ORDER BY DESC(?entries)
LIMIT 10
```

Songs Explorer — filtered and paginated retrieval

```
SELECT ?song ?songname ?mainArtist ?artistname ?genre ?energy ?danceability
?valence
WHERE {
  {
    SELECT DISTINCT ?song ?songname ?mainArtist ?artistname ?energy ?danceability
    ?valence
    WHERE {
      ?song a type:Song ; pred:name ?songname ; pred:mainArtist ?mainArtist .
      ?mainArtist pred:name ?artistname .
      OPTIONAL { ?song pred:energy ?energy . }
      OPTIONAL { ?song pred:danceability ?danceability . }
      OPTIONAL { ?song pred:valence ?valence . }
      OPTIONAL { ?song pred:genre ?genre . }
      # optional FILTERs for song, artist, genre, popularity
    }
    ORDER BY DESC(?energy)
    LIMIT 50
    OFFSET 0
  }
  OPTIONAL { ?song pred:genre ?genre . }
}
```

This query shape avoids limiting duplicated rows caused by songs that have multiple genres.

Insights — party score

```
SELECT ?song ?songName ?mainArtist ?artistName ?partyScore
WHERE {
  ?song a type:Song ;
    pred:name ?songName ;
    pred:mainArtist ?mainArtist ;
    pred:energy ?energy ;
    pred:danceability ?danceability ;
    pred:valence ?valence .
  ?mainArtist pred:name ?artistName .
}
```

```

    BIND((?energy + ?danceability + ?valence) / 3 AS ?partyScore)
  }
ORDER BY DESC(?partyScore)
LIMIT 25

```

Advanced analytical queries used in Insights

The following queries are more complex because they combine aggregation, grouping, optional relations, ordering rules, and date transformations.

Most Versatile Artists (distinct genres per artist)

```

SELECT ?artist ?artistName (COUNT(DISTINCT ?genre) AS ?genreCount)
WHERE {
  ?song a type:Song ; pred:mainArtist ?artist ; pred:genre ?genre .
  ?artist pred:name ?artistName .
}
GROUP BY ?artist ?artistName
ORDER BY DESC(?genreCount) ?artistName
LIMIT 20

```

Most Resilient Songs (chart longevity + best rank)

```

SELECT ?song ?songName ?artistName
      (MAX(?weeks) AS ?maxWeeks)
      (MIN(?rank) AS ?bestRank)
      (COUNT(?entry) AS ?entries)
WHERE {
  ?entry a type:ChartEntry ;
    pred:song ?song ;
    pred:rank ?rank ;
    pred:weeks ?weeks .
  ?song pred:name ?songName ; pred:mainArtist ?artist .
  ?artist pred:name ?artistName .
}
GROUP BY ?song ?songName ?artistName
ORDER BY DESC(?maxWeeks) ASC(?bestRank) DESC(?entries)
LIMIT 30

```

Seasonality (entries by month over all years)

```

SELECT ?month (COUNT(?entry) AS ?entries)
WHERE {
  ?entry a type:ChartEntry ; pred:date ?date .
  BIND(MONTH(xsd:date(?date)) AS ?month)
}
GROUP BY ?month
ORDER BY ?month

```

Top Collaboration Pairs (main artist + featured artist)

```

SELECT ?artistAName ?artistBName
      (COUNT(DISTINCT ?song) AS ?collabCount)
      (GROUP_CONCAT(DISTINCT ?songName; separator=", ") AS ?examples)
WHERE {
  ?song a type:Song ;
    pred:mainArtist ?a1 ;
    pred:featuredArtist ?a2 ;
    pred:name ?songName .
  ?a1 pred:name ?a1Name .
  ?a2 pred:name ?a2Name .
  FILTER(?a1 != ?a2)
  BIND(IF(STR(?a1Name) < STR(?a2Name), ?a1Name, ?a2Name) AS ?artistAName)

```

```

    BIND(IF(STR(?a1Name) < STR(?a2Name), ?a2Name, ?a1Name) AS ?artistBName)
  }
GROUP BY ?artistAName ?artistBName
ORDER BY DESC(?collabCount)
LIMIT 20

```

Update Operations (INSERT / DELETE / DELETE+INSERT)

SPARQL Update is used in the Operations page to add, edit, and remove songs, artists, genres, attributes, and chart entries.

INSERT — add alternative genre to a song

```

INSERT {
  ?song pred:genre "alt-pop" .
}
WHERE {
  ?song a type:Song ; pred:name "Blinding Lights" .
}

```

DELETE WHERE — remove featured artist relationship

```

DELETE WHERE {
  ?song a type:Song ;
    pred:name "Old Town Road" ;
    pred:featuredArtist ?feat .
}

```

DELETE + INSERT — replace an attribute value safely

```

DELETE { ?song pred:valence ?old . }
INSERT { ?song pred:valence 0.49 . }
WHERE {
  ?song a type:Song ; pred:name "Anti-Hero" .
  OPTIONAL { ?song pred:valence ?old . }
}

```

Validation and Quality Queries

To support maintenance and debugging, we also run quality checks:

- songs without main artist
- chart entries without rank or date
- potential duplicates with same normalized title and main artist

These checks were important while fixing collaboration parsing and matching errors, especially in artist names containing separators that can represent either collaborations or official band names.

Implementation Notes

- OPTIONAL is used for partial enrichment fields (some songs have no Spotify attributes).
- Pagination is implemented with LIMIT/OFFSET and a separate COUNT(DISTINCT ?song) query.
- Multi-valued fields (genre) can produce repeated rows and are deduplicated in application code by song URI.
- Update operations return explicit success/error messages in the UI for safer manual data management.

Application Features (UI)

The interface is organized around exploration and navigation between linked entities. Users can move from aggregate views (Dashboard and Insights) to filtered lists (Songs) and then to detailed entity pages (Song and Artist), always through direct links.

Dashboard (/)

Entry page of the application. Displays overall dataset statistics: total number of songs, artists, and chart entries. Shows the top 10 artists by number of chart entries and the top 10 songs by Spotify popularity, each with a link to their detail page.

Songs Explorer (/songs/)

Paginated list of songs (50 per page) with text-based filters for song name, artist name, and genre (case-insensitive substring matching), plus optional popularity range and metric-based sorting. Results show the song title (link to detail), the main artist (link to artist detail), genre badges, and audio attributes (energy, danceability, valence). The total number of matching songs is shown above the table, and pagination preserves active filters across pages.

Song Detail (/songs/<id>/?uri=...)

Detailed view of a single song. Displays:

- Title, album name, duration (formatted as m:ss), explicit flag
- Main artist and featured artists, each linking to their artist detail page
- Genre tags
- Spotify popularity and tempo
- Seven audio features rendered as labelled progress bars (0–1 scale): energy, danceability, valence, acousticness, speechiness, instrumentality, liveness
- An embedded Spotify player when a spotifyTrackId is stored in the triplestore
- A chart history table (up to 50 entries): rank, weeks on chart, date
- A "Similar Songs" section showing the 6 songs with the smallest Manhattan distance across the same audio features

Artist Detail (/artists/<id>/?uri=...)

Profile page for an artist. Displays:

- Artist name, associated genres, total songs as main artist, total chart entries, best chart rank, average energy, average danceability, average valence, and collaborator count
- A list of the artist's songs (name, genre, popularity, energy, danceability), each linking to song detail
- A collaboration graph: an SVG visualization placing the artist at the centre and the top 12 collaborators (by a score combining shared songs and chart entry date overlap) arranged in a circle, with tooltips showing shared track names and counts

SPARQL Operations (/operations/)

Data management page with six groups of forms, one for each entity type and sub-operation:

- Songs: add a new song (with main artist and optional genre), rename, delete (with cascade confirmation if linked songs exist)

- Genres: add a genre to a song, rename, remove
- Attributes: add, edit, or remove an audio attribute on a song, restricted to the allowed attribute list
- Artists: add a new artist, rename, delete (with cascade option)
- Chart Entries: add a new entry (rank, weeks, date), edit an existing entry, remove an entry by URI

All form dropdowns (song names, artist names, genres, chart entry URIs) are populated from the triplestore and cached in Django's memory cache for 60 seconds. Every operation returns a clear success or error message.

Insights (/insights/)

Analytical page with six sections derived from more complex SPARQL queries:

- **Party Score** — top 25 songs ranked by the average of energy, danceability, and valence
- **Hidden Gems** — songs with energy ≥ 0.75 , danceability ≥ 0.75 , and up to, at most, 2 chart entries
- **Most Versatile Artists** — top 20 artists ranked by number of distinct genres across their songs
- **Most Resilient Songs** — top 30 songs ranked by peak weeks on chart and best rank
- **Seasonality** — bar chart of chart entries by month across all years
- **Top Collaboration Pairs** — top 20 artist pairs most frequently appearing together as main artist and featured artist

Billboard (/billboard/)

Historical chart viewer. Presents a date tree (year $\rightarrow$ month $\rightarrow$ day) built from all distinct dates in the triplestore. Selecting a date shows the full chart for that week: rank, weeks on chart, song (link to detail), and artist (link to detail). Defaults to the most recent available date.

About Data (/about-data/)

Informational page documenting data sources, the normalization pipeline (CSV $\rightarrow$ RDF), main namespaces, and instructions for importing into GraphDB.

Conclusions

The project achieved its proposed objectives: building an RDF dataset from heterogeneous sources, semantically representing relationships among songs, artists, and charts, and developing a web application that explores this dataset through SPARQL.

The chosen data model, with ChartEntry as an independent entity, proved suitable for representing the temporal nature of the data (the same song can have dozens of entries on different dates) without duplicating information.

The normalization and matching pipeline between both datasets was the most challenging component, requiring particular attention to variability in collaboration representation and to managing false positives when splitting band names that contain connectors such as "&" or "and".

Integration with Spotify significantly enriched the dataset with audio attributes, enabling more interesting analyses on the Insights page.

Setup to Run the Application

Make sure to have GraphDB up and running on port 7200. Then, go to the project root. To just build and load the data into GraphDB, run `./build.sh`. To build and then run the django app, run `./build_and_run.sh`. To simply run the Django app, after it is built, run `./run_app.sh`. The app is on the Django url, <http://127.0.0.1:8000/>.